Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07

ПРАКТИЧЕСКАЯ РАБОТА № 7

Дисциплина МДК 03.02

Анализ угроз и уязвимостей

Вариант № 2
АИС «Умный кампус»
Модуль Library System

Студент: Шваб Роберт Эдуардович
Группа: И13_24_11
Подпись: _____

Руководитель: _____
Дата представления: «___» _____ 2026 г.

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

1 ВЫПОЛНЕНИЕ РАБОТЫ.....3

1.1 Движение данных и границы доверия.....3

1.2 Модель STRIDE.....4

1.3 Эшелонированная защита.....5

1.4 Выводы.....6

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....7

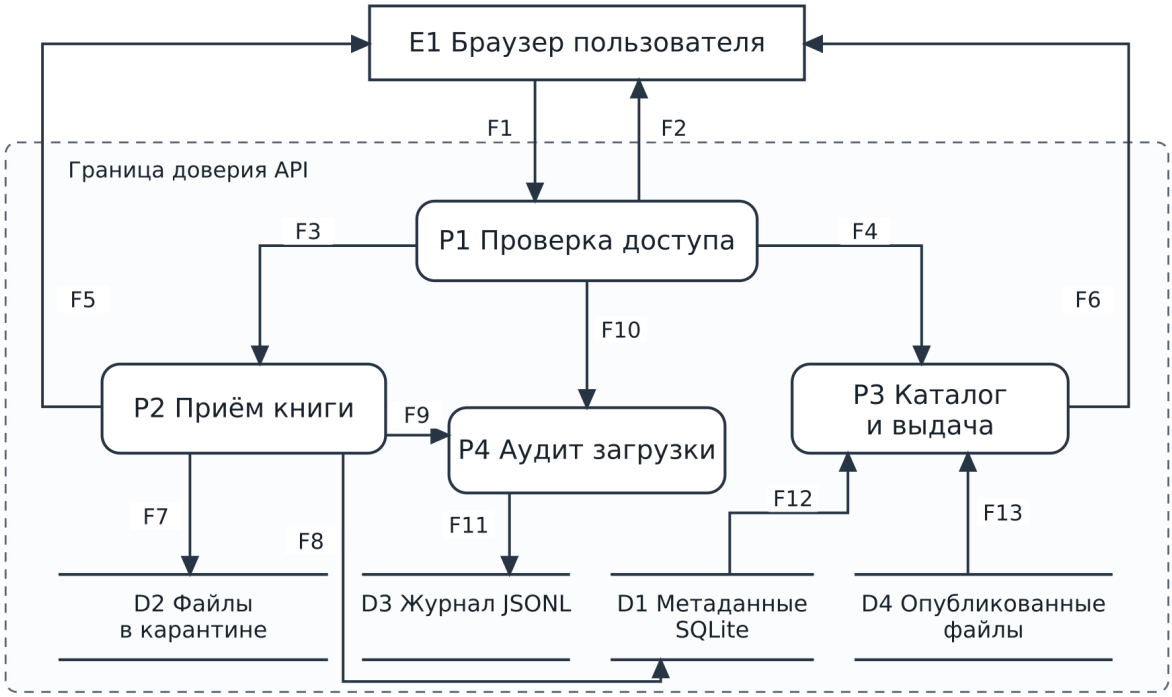
ПРИЛОЖЕНИЕ А Листинг исходного кода.....8

1 ВЫПОЛНЕНИЕ РАБОТЫ

Я выполнил работу по варианту 2 Library System АИС «Умный кампус» на C# и .NET 8. Модуль принимает PDF/EPUB/ZIP в карантин; права Reader и Librarian разделены [2]. Оформление выполнено по положению института [1].

1.1 Движение данных и границы доверия

На рисунке 1 показаны внешняя сущность E1 (браузер), процессы P1–P4 и четыре хранилища: D1 — метаданные SQLite, D2 — файлы карантина, D3 — журнал JSONL, D4 — опубликованные файлы. Пунктир отделяет браузер от серверной части. Стрелки F1–F13 показывают направление передачи данных; расшифровка дана в таблице 1.



Источник: составлено автором

Рисунок 1 — DFD приёма и выдачи электронных книг

Имя файла, MIME, содержимое и заголовок Authorization считаются недоверенными до серверной проверки. Читатель не получает прямого пути. Карантин расположен вне статического веб-каталога; выдача использует

отдельную проверку Published. P4 обозначает функцию middleware аудита, а не отдельный сетевой сервис.

Таблица 1 — Именованные потоки DFD

Поток	Передаваемые данные
F1	Токен, запрос каталога или файл с названием книги
F2	Отказ на этапе проверки доступа: HTTP 401 или 403
F3	Разрешённая загрузка: субъект, название и файл
F4	Субъект, строка поиска или идентификатор книги
F5	Результат загрузки: HTTP 201 с метаданными или ошибка
F6	Каталог, опубликованный файл или ошибка запроса
F7	Проверенные байты книги и серверное имя файла
F8	Метаданные карантина, владелец и SHA-256
F9	Код ответа операции загрузки и идентификатор запроса
F10	Сведения об отказе в загрузке на этапе доступа
F11	Запись аудита: requestId, actor, status и время
F12	Метаданные каталога и сведения о публикации
F13	Байты только опубликованного файла

Источник: составлено автором по результатам работы

1.2 Модель STRIDE

Я рассмотрел все шесть категорий STRIDE. OWASP указан по редакции 2025 года; это сопоставление близких рисков, а не формальное равенство классификаций [3].

Сведения представлены в таблице 2.

Таблица 2 — Матрица угроз STRIDE

Категория	Угроза и узел	OWASP 2025	Защита
S · Spoofing	Подмена роли библиотекаря; P1	A07:2025 Authentication Failures	Серверный principal и случайный токен
T · Tampering	Перезапись файла через ZIP; P2, D2	A01:2025 Broken Access Control	Проверка пути, UUID и приватный карантин
R · Repudiation	Отрицание запрещённой загрузки; P4, D3	A09:2025 Security Logging and Alerting Failures	Журнал requestId, actor, status без токенов
I · Information Disclosure	Чтение чужого карантина; P3, D1, D2	A01:2025 Broken Access Control	Фильтр Published / Owner, нет прямых путей
D · Denial of Service	ZIP-бомба и частые загрузки; P2	A06:2025 Insecure Design	Размеры, сжатие, число записей и Rate Limiting
E · Elevation of Privilege	Reader выполняет запись; P1, P2	A01:2025 Broken Access Control	Authorize LibrarianOnly и отрицательный тест

Источник: анализ модуля автором; категории OWASP по [3]

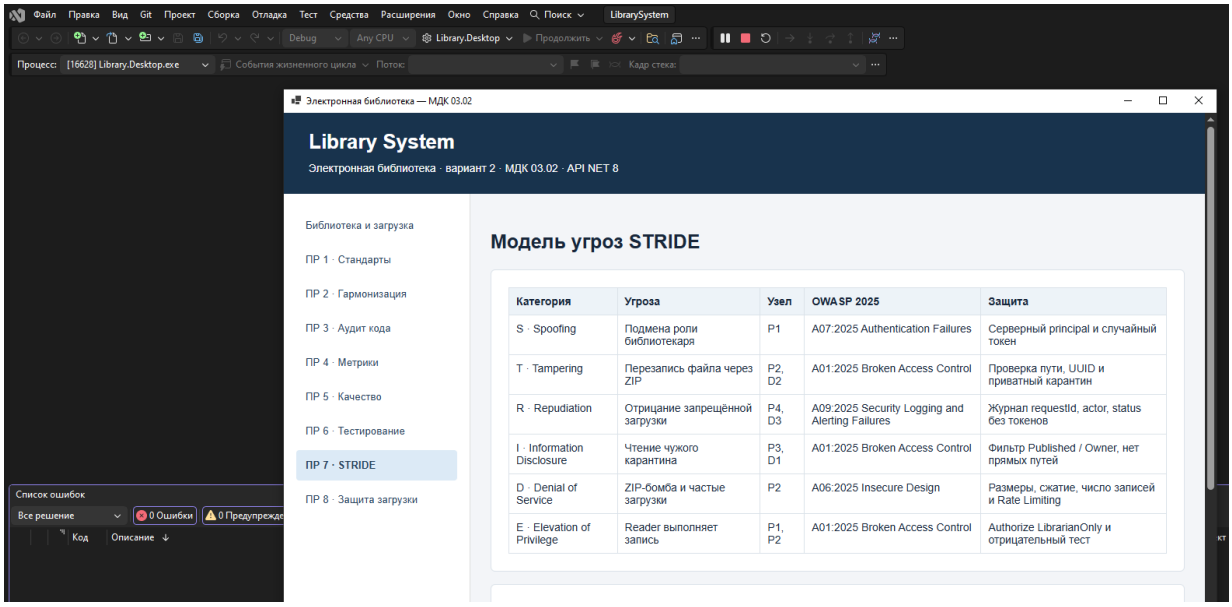
1.3 Эшелонированная защита

Серверная аутентификация и LibrarianOnly образуют первый барьер. FluentValidation и проверка содержимого — следующий. Для ZIP затем проверяются пути, ссылки, дубли, размер и сжатие; сервер назначает UUID. Последний барьер — карантин: даже допустимый документ не становится опубликованным автоматически. OWASP рекомендует сочетать независимые меры загрузки [4].

EF Core параметризует поиск; контролируемые ошибки не раскрывают пути. Журнал связывает операцию с requestId и субъектом без записи токена. Локальный журнал не защищён от администратора: для эксплуатации нужен удалённый сбор с контролем целостности. Ограничения запросов не заменяют предел распаковки — это разные способы исчерпания ресурсов.

Остаточные риски: вредоносный PDF может иметь корректную сигнатуру; внешняя идентификация требует доверенного IdP; loopback HTTP не моделирует защищённый внешний транспорт. Перед внешним размещением нужны HTTPS и AV/CDR перед публикацией. Они не приписываются уже выполненному модулю. Матрица и схема показаны на вкладке «ПР 7 · STRIDE». Исходная схема доступна в DFD.svg.

На рисунке 2 показана матрица угроз STRIDE. Схема потоков данных приведена отдельно на рисунке 1.



Источник: снимок прогона на Windows, предоставленный автором.

Рисунок 2 — Матрица угроз STRIDE в приложении

1.4 Выводы

Я связал DFD, шесть категорий STRIDE и меры защиты. Независимые барьеры не позволяют одной пропущенной проверке автоматически превратиться в выдачу недоверенного файла читателю.

Полные исходники, входные данные и журналы переданы с отчётом. Основной листинг приведён в приложении А.

Исходный код и порядок запуска: репозиторий MDK0302-LibrarySystem.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Положение об оформлении письменных работ. Санкт-Петербургский институт экономики и управления. Методические материалы [1].

2. Практические работы МДК 03.02. Задания ПР 1–8 и эссе. Вариант 2 Library System. Материалы преподавателя [2].

3. OWASP Top 10:2025. Application Security Risks. — URL: https://top10.owasp.org/2025/ (дата обращения: 08.11.2026) [3].

4. OWASP. File Upload Cheat Sheet. — URL: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html (дата обращения: 08.11.2026) [4].

ПРИЛОЖЕНИЕ A
Листинг исходного кода

src/Library.Tools/Program.cs

```
using System.Diagnostics;
using System.Text;
using System.Text.Json;
using System.Text.Encodings.Web;
using System.Xml.Linq;
using Library.Tools;
using Library.Core;
Console.OutputEncoding = Encoding.UTF8;
var directory = new DirectoryInfo(Directory.GetCurrentDirectory());
while (directory is not null && !Directory.Exists(Path.Combine(directory.FullName, "artifacts"))) directory =
directory.Parent;
string root = directory?.FullName ?? throw new IOException("Запуск из корня Project");
string verification = Path.Combine(root, "verification");Directory.CreateDirectory(verification);
var options = new JsonSerializerOptions(JsonSerializerDefaults.Web) { WriteIndented = true, Encoder =
JavaScriptEncoder.UnsafeRelaxedJsonEscaping };
void Save(string name, object data) => File.WriteAllText(Path.Combine(verification, name + ".json"),
JsonSerializer.Serialize(data, options));
string command = args.FirstOrDefault() ?? "all";
try
{
  if (command is "all" or "samples")
  {
    Directory.CreateDirectory(Path.Combine(root, "samples"));
    File.WriteAllBytes(Path.Combine(root, "samples/sample.pdf"), Samples.Pdf());
    File.WriteAllBytes(Path.Combine(root, "samples/sample.epub"), Samples.Epub());
    File.WriteAllBytes(Path.Combine(root, "samples/zip-slip.zip"), Samples.Zip(("../escape.pdf",
Samples.Pdf())));
  }
  foreach (string name in new[] { "standards", "harmonization", "quality", "threats", "bugs" })
    if (command == "all" || command == name)
    {
      string text = File.ReadAllText(Path.Combine(root, "artifacts", name + ".json"));
      File.WriteAllText(Path.Combine(verification, name + ".json"), text);
      var rows = JsonDocument.Parse(text).RootElement;
      string Escape(string s) => "\"" + s.Replace("\"", "\"\"") + "\"";
      var lines = new List<string> { string.Join(';', rows[0].EnumerateObject().Select(x => Escape(x.Name))) };
      lines.AddRange(rows.EnumerateArray().Select(row => string.Join(';', row.EnumerateObject().Select(x
=> Escape(x.Value.ToString())))));
      File.WriteAllLines(Path.Combine(verification, name + ".csv"), lines, new UTF8Encoding(true));
      Console.WriteLine(name + ": строк " + rows.GetArrayLength());
    }
  if (command is "all" or "audit" or "metrics")
  {
    string before = File.ReadAllText(Path.Combine(root, "before/LegacyUploadController.cs"));
    string after = File.ReadAllText(Path.Combine(root, "src/Library.Api/UploadController.cs"));
    string catalog = File.ReadAllText(Path.Combine(root, "src/Library.Api/BooksController.cs"));
    var result = new { before = CodeQuality.Measure(before, "LegacyUploadController"),
      after = CodeQuality.Measure(after, "UploadController"), catalog = CodeQuality.Measure(catalog,
"BooksController"),
      beforeAudit = CodeQuality.Audit(before), afterAudit = CodeQuality.Audit(after), catalogAudit =
CodeQuality.Audit(catalog) };
    Save("analysis", result);Console.WriteLine(JsonSerializer.Serialize(result, options));
  }
```

```
  if (command == "parse-tests")
  {
    string path = args.Length > 1 ? args[1] : Path.Combine(verification, "library-tests.trx");
    var tests = XDocument.Load(path).Descendants().Where(x => x.Name.LocalName == "UnitTestResult")
      .Select(x => new { name = (string?)x.Attribute("testName"), outcome = (string?)x.Attribute("outcome"),
duration = (string?)x.Attribute("duration") }).OrderBy(x => x.name).ToArray();
    if (tests.Length == 0) throw new IOException("Нет результатов TRX");
    var result = new { total = tests.Length, passed = tests.Count(x => x.outcome == "Passed"), failed =
tests.Count(x => x.outcome == "Failed"), skipped = tests.Count(x => x.outcome is not ("Passed" or "Failed")),
tests };
    Save("tests", result);Console.WriteLine($"Всего {result.total}; Passed {result.passed}; Failed
{result.failed}; Skipped {result.skipped}");
    if (result.failed > 0 || result.skipped > 0) Environment.ExitCode = 1;
  }
  if (command == "api-checks")
  {
    using var client = new HttpClient { BaseAddress = new Uri(args.Length > 1 ? args[1] :
"http://127.0.0.1:5352") };
    using var response = await client.PostAsync("/lab/scenarios", null);response.EnsureSuccessStatusCode();
    string text = await response.Content.ReadAsStringAsync();File.WriteAllText(Path.Combine(verification,
"api-scenarios.json"), text);Console.WriteLine(text);
    if (JsonDocument.Parse(text).RootElement.EnumerateArray().Any(x => !
x.GetProperty("passed").GetBoolean())) Environment.ExitCode = 1;
  }
  if (command == "open-ui")
  {
    var url = new Uri(args[1]);using var client = new HttpClient { Timeout = TimeSpan.FromSeconds(2) };bool
ready = false;
    for (int i = 0; i < 30 && !ready; i++)
    {
      try { using var response = await client.GetAsync(new Uri(url, "/health"));ready =
response.IsSuccessStatusCode; }
      catch (HttpRequestException) { } catch (TaskCanceledException) { }
      if (!ready) await Task.Delay(500);
    }
    if (!ready) throw new IOException("Сервер ещё не запущен");
    Process.Start(new ProcessStartInfo(url.AbsoluteUri) { UseShellExecute = true });
  }
}
catch (Exception e) { Console.Error.WriteLine("Операция не выполнена: " +
e.Message);Environment.ExitCode = 1; }
```

src/Library.Api/Program.cs

```
using System.Net;
using System.Security.Claims;
using System.Text.Json;
using System.Threading.RateLimiting;
using Library.Api;
using Library.Core;
using Microsoft.AspNetCore.Authentication;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);
string storage = builder.Configuration["StorageRoot"] ??
Path.Combine(builder.Environment.ContentRootPath, "App_Data");
Directory.CreateDirectory(storage);
Directory.CreateDirectory(Path.Combine(storage, "quarantine"));
Directory.CreateDirectory(Path.Combine(storage, "published"));
builder.Services.AddSingleton<StorageRoot>(services => new StorageRoot(
```

```
    services.GetRequiredService<IConfiguration>()["StorageRoot"] ??
Path.Combine(builder.Environment.ContentRootPath, "App_Data")));
builder.Services.AddSingleton<TokenRegistry>();
builder.Services.AddSingleton<MetadataValidator>();
builder.Services.AddScoped<UploadService>();
builder.Services.AddDbContext<LibraryDb>(o => o.UseSqlite("Data Source=" + Path.Combine(storage,
"library.db")));
builder.Services.AddControllers();
builder.Services.AddAuthentication("LabToken").AddScheme<AuthenticationSchemeOptions,
TokenHandler>("LabToken", null);
builder.Services.AddAuthorization(o => o.AddPolicy("LibrarianOnly", p =>
p.RequireAuthenticatedUser().RequireRole("Librarian")));
builder.Services.AddRateLimiter(o => {
    o.RejectionStatusCode = 429;
    o.AddPolicy("uploads", context => RateLimitPartition.GetFixedWindowLimiter(
        context.User.FindFirstValue(ClaimTypes.NameIdentifier) ?? "anonymous", _ => new
FixedWindowRateLimiterOptions
        { PermitLimit = 3, Window = TimeSpan.FromMinutes(1), QueueLimit = 0 }));
});
var app = builder.Build();
storage = app.Services.GetRequiredService<StorageRoot>().Path;
Directory.CreateDirectory(storage);
Directory.CreateDirectory(Path.Combine(storage, "quarantine"));
Directory.CreateDirectory(Path.Combine(storage, "published"));
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<LibraryDb>();
    db.Database.EnsureCreated();
    if (!db.Books.Any())
    {
        byte[] sample = Samples.Pdf();
        File.WriteAllBytes(Path.Combine(storage, "published", "sample.pdf"), sample);
        db.Books.Add(new Book { Id = Guid.Parse("11111111-1111-4111-8111-111111111111"), Title = "Учебный
документ библиотеки",
            OriginalName = "sample.pdf", StoredName = "sample.pdf", Owner = "system", Published = true, Length
= sample.Length });
        db.SaveChanges();
    }
}
var auditLock = new object();
app.Use(async (context, next) => {
    try { await next(); }
    catch (RejectedFile error) {
        context.Response.StatusCode = error.Status;
        await context.Response.WriteAsJsonAsync(new { code = "upload_rejected", message = error.Message });
    }
    catch (Exception error) {
        app.Logger.LogError(error, "Ошибка запроса {RequestId}", context.TraceIdentifier);
        context.Response.StatusCode = 500;
        await context.Response.WriteAsJsonAsync(new { code = "server_error", requestId =
context.TraceIdentifier });
    }
    finally {
        if (context.Request.Path == "/api/books/upload") lock (auditLock)
            File.AppendAllText(Path.Combine(storage, "audit.jsonl"), JsonSerializer.Serialize(new {
                requestId = context.TraceIdentifier, actor =
context.User.FindFirstValue(ClaimTypes.NameIdentifier),
                status = context.Response.StatusCode, at = DateTimeOffset.UtcNow }) + Environment.NewLine);
    }
```

```
});
app.UseAuthentication();app.UseAuthorization();app.UseRateLimiter();
app.MapControllers();app.MapGet("/health", () => Results.Ok(new { status = "ok" }));
LabEndpoints.Map(app);
app.Run();
public partial class Program { }
```

src/Library.Api/Security.cs

```
using System.Collections.Concurrent;
using System.Security.Claims;
using System.Security.Cryptography;
using System.Text.Encodings.Web;
using Microsoft.AspNetCore.Authentication;
using Microsoft.Extensions.Options;
namespace Library.Api;

/// <summary>Случайные серверные токены стенда без клиентского назначения роли.</summary>
public sealed class TokenRegistry
{
  private readonly ConcurrentDictionary<string, ClaimsPrincipal> _tokens = new();
  public string Issue(string role, string actor)
  {
    string token = Convert.ToHexString(RandomNumberGenerator.GetBytes(32));
    _tokens[token] = new ClaimsPrincipal(new ClaimsIdentity(new[] {
      new Claim(ClaimTypes.NameIdentifier, actor), new Claim(ClaimTypes.Role, role) }, "LabToken"));
    return token;
  }
  public ClaimsPrincipal? Find(string token) => _tokens.GetValueOrDefault(token);
}
public sealed class TokenHandler(IOptionsMonitor<AuthenticationSchemeOptions> options,
  ILoggerFactory logger, UrlEncoder encoder, TokenRegistry tokens)
  : AuthenticationHandler<AuthenticationSchemeOptions>(options, logger, encoder)
{
  protected override Task<AuthenticateResult> HandleAuthenticateAsync()
  {
    string value = Request.Headers.Authorization.ToString();
    var principal = value.StartsWith("Bearer ") ? tokens.Find(value[7..]) : null;
    return Task.FromResult(principal is null ? AuthenticateResult.NoResult() :
      AuthenticateResult.Success(new AuthenticationTicket(principal, Scheme.Name)));
  }
}
```